```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
  <title>DeepSeek Chat</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
      background: #f5f5f5;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
    }
    .chat-container {
      width: 100%;
      max-width: 600px;
      height: 100vh;
      display: flex;
      flex-direction: column;
      background: #fff;
      box-shadow: 0 0 20px rgba(0,0,0,0.1);
    }
    .header {
      padding: 16px;
      background: #4f46e5;
      color: white;
      text-align: center;
      font-size: 18px;
      font-weight: bold;
    }
    .messages {
      flex: 1;
      overflow-y: auto;
      padding: 16px;
      display: flex;
      flex-direction: column;
```

```
  gap: 12px;
}
.message {
  max-width: 80%;
  padding: 10px 14px;
  border-radius: 18px;
  line-height: 1.5;
  word-wrap: break-word;
  white-space: pre-wrap;
}
.user-msg {
  align-self: flex-end;
  background: #4f46e5;
  color: white;
  border-bottom-right-radius: 4px;
}
.assistant-msg {
  align-self: flex-start;
  background: #e5e7eb;
  color: #1f2937;
  border-bottom-left-radius: 4px;
}
.input-area {
  display: flex;
  padding: 12px;
  border-top: 1px solid #ddd;
  background: #fafafa;
}
.input-area input {
  flex: 1;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 24px;
  font-size: 16px;
  outline: none;
}
.input-area button {
  margin-left: 8px;
  padding: 0 20px;
  border: none;
  background: #4f46e5;
```

```
      color: white;
      border-radius: 24px;
      font-size: 16px;
      cursor: pointer;
      white-space: nowrap;
    }
    .input-area button:disabled {
      background: #a5b4fc;
      cursor: not-allowed;
    }
    .notice {
      text-align: center;
      color: #999;
      font-size: 12px;
      padding: 4px;
    }
  </style>
</head>
<body>
  <div class="chat-container">
    <div class="header">🤖 DeepSeek 助手</div>
    <div class="messages" id="messages"></div>
    <div class="input-area">
      <input type="text" id="userInput" placeholder="输入消息..."
autocomplete="off" />
      <button id="sendBtn">发送</button>
    </div>
    <div class="notice">API Key 仅用于个人测试，请勿分享页面</div>
  </div>

  <script>
    // ********** 把你的 API Key 填在这里 **********
    const API_KEY = 'sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx';
    const API_URL = 'https://api.deepseek.com/v1/chat/completions';

    const messagesDiv = document.getElementById('messages');
    const userInput = document.getElementById('userInput');
    const sendBtn = document.getElementById('sendBtn');

    // 保存对话历史
    let conversationHistory = [
```

```
  { role: 'system', content: '你是一个有帮助的助手' }
];

// 添加一条消息到界面
function appendMessage(role, text) {
  const msgDiv = document.createElement('div');
  msgDiv.classList.add('message', role === 'user' ? 'user-msg' : 'assistant-msg');
  msgDiv.textContent = text;
  messagesDiv.appendChild(msgDiv);
  messagesDiv.scrollTop = messagesDiv.scrollHeight;
  return msgDiv;
}

// 发送请求（流式）
async function sendMessage(userText) {
  // 添加用户消息
  conversationHistory.push({ role: 'user', content: userText });
  appendMessage('user', userText);
  userInput.value = '';
  sendBtn.disabled = true;

  // 创建一条空的助手消息，后续流式填充
  const assistantMsgDiv = appendMessage('assistant', '');
  let fullContent = '';

  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${API_KEY}`
      },
      body: JSON.stringify({
        model: 'deepseek-chat',
        messages: conversationHistory,
        stream: true
      })
    });

    if (!response.ok) {
      const error = await response.json();
```

```
    throw new Error(error.error?.message || `HTTP ${response.status}`);
  }

  // 读取流数据
  const reader = response.body.getReader();
  const decoder = new TextDecoder();
  let buffer = '';

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    buffer += decoder.decode(value, { stream: true });
    // 按行分割
    const lines = buffer.split('\n');
    buffer = lines.pop(); // 保留未完成的最后一部分

    for (const line of lines) {
      const trimmed = line.trim();
      if (!trimmed || !trimmed.startsWith('data: ')) continue;
      const dataStr = trimmed.slice(6);
      if (dataStr === '[DONE]') continue;

      try {
        const json = JSON.parse(dataStr);
        const delta = json.choices?.[0]?.delta?.content;
        if (delta) {
          fullContent += delta;
          assistantMsgDiv.textContent = fullContent;
          messagesDiv.scrollTop = messagesDiv.scrollHeight;
        }
      } catch (e) {
        // 忽略解析错误
      }
    }
  }

  // 将完整回复加入历史
  conversationHistory.push({ role: 'assistant', content: fullContent });
} catch (error) {
  assistantMsgDiv.textContent = `❌ 错误: ${error.message}`;
```

```
            // 移除刚加入的用户消息（避免历史错乱）
            conversationHistory.pop();
        } finally {
            sendBtn.disabled = false;
            userInput.focus();
        }
    }

    // 事件绑定
    sendBtn.addEventListener('click', () => {
        const text = userInput.value.trim();
        if (text) sendMessage(text);
    });

    userInput.addEventListener('keypress', (e) => {
        if (e.key === 'Enter') {
            const text = userInput.value.trim();
            if (text) sendMessage(text);
        }
    });
  </script>
</body>
</html>
```